

Avaliação Técnica: Desenvolvedor(a) Fullstack Pleno

Objetivo

Avaliar a organização de código, o consumo de APIs externas e a habilidade de estruturar uma aplicação Fullstack funcional, segura e bem estilizada.

Formato e Prazos

Perguntas teóricas: 4 questões incluídas no README.

Desafio Prático: Implementação de um CRUD com integração de API pública.

Prazo sugerido: Até 4 dias corridos.

Tempo estimado: De 4 a 8 horas de execução.

Uso de IA: Permitido como ferramenta de auxílio. O foco da avaliação será a sua capacidade de validar, adaptar e organizar o código gerado.

1. Raciocínio Técnico (Perguntas Abertas)

Responda de forma objetiva no seu README.md final:

Idempotência: Como você garante que um usuário não cadastre o mesmo item duas vezes ao clicar rapidamente no botão "Enviar"?

Segurança: Quais são as 3 principais medidas que você toma para proteger uma API Node.js?

Estado no Front-end: Em que situações você prefere usar Context API em vez de apenas passar propriedades (props) entre componentes?

Manutenibilidade: Como você organiza seus arquivos de CSS (puro ou modules) para que eles não fiquem confusos conforme o projeto cresce?

2. Desafio Prático

Cenários de Negócio (Escolha APENAS UM)

Implemente uma aplicação que permita listar e cadastrar dados, integrando obrigatoriamente com uma API externa:

Cenário	O que construir	Sugestão de API
---------	-----------------	-----------------

Opção A: Consulta de CEP	Cadastro de contatos onde, ao digitar o CEP, os campos de endereço são preenchidos automaticamente.	ViaCEP
Opção B: Catálogo de Filmes	Cadastro de filmes favoritos, onde o usuário busca o título e o sistema traz dados como sinopse e capa.	TMDB ou OMDb
Opção C: Gestão de Clima	Gerenciador de tarefas/cidades que salva o local do usuário e exibe a temperatura atual em tempo real.	OpenWeather

Requisitos Técnicos

Back-End (Node.js)

Arquitetura: Organização clara entre rotas e lógica de negócio (ex: camadas de Controller/Service).

Banco de Dados: com banco relacional PostgreSQL ou MySQL.

Integração: Realizar a chamada para a API externa escolhida e persistir os dados necessários no seu banco.

Front-End (React)

Estilização: Utilize **CSS puro ou CSS Modules**.

Layout: A interface deve ser organizada e responsiva (Flexbox/Grid).

Formulário: Validação básica de campos obrigatórios.

Listagem: Exibição dos dados salvos no banco de forma clara.

Infraestrutura (Docker)

Orquestração: O projeto deve subir inteiramente com o comando `docker-compose up`.

Serviços: O compose deve contemplar a aplicação (Back e Front) e o banco de dados.

Critérios de Avaliação

Domínio de CSS: Capacidade de criar uma interface agradável e organizada usando fundamentos de CSS.

Organização de Código: Código legível, nomes de variáveis semânticos e estrutura de pastas lógica.

Consumo de API: Tratamento correto de respostas e possíveis erros na comunicação com a API externa.

Pragmatismo: O sistema deve ser funcional e fácil de rodar.

Entregáveis

Código-fonte: Link do repositório público (GitHub ou GitLab).

README.md contendo:

Instruções de como rodar o projeto via Docker.

Respostas das 4 perguntas teóricas da Parte 1.

Breve explicação das decisões técnicas tomadas.

Auto-crítica: O que você faria de diferente ou melhoraria se tivesse mais tempo?

Boa sorte! Estamos ansiosos para ver sua solução.